

基于 CUDA 小矩阵加速的机械臂批量逆运动学求解

匿名稿（通讯作者信息在单独标题页）

（作者单位信息在单独标题页）

文章编号：（预留） DOI：（预留）

摘要：针对机械臂批量逆运动学求解中 6×6 小矩阵反复求解与多 kernel 调度带来的固定开销问题，提出基于计算统一设备架构（compute unified device architecture, CUDA）底层优化的批量阻尼最小二乘法加速方法。将每个目标映射为 1 个 CUDA 线程块，在单次 kernel launch 中完成全部迭代；设计寄存器驻留的 6×6 LDL^T 求解器替代 cuBLAS；采用 FP32/FP64 混合精度策略。以官方 UR10 为对象构建统一 benchmark，从端到端、GPU stream 和有效吞吐三个维度报告性能并引用 cuRobo（CUDA Graph 开启）为公平基线。实验表明：当 $N \geq 500$ 时本文方法吞吐稳定在 128k–157k targets/s，GPU 时间线性扩展（ $R^2 > 0.999$ ）；在 $N = 100$ 极小批量下 GPU 流延迟较 cuRobo-Graph 降低约 52%。当 $N \geq 500$ 时 cuRobo-Graph 依靠 Graph 和批处理优化通常获得更高吞吐，但其存在批量依赖的非单调退化。本文方法的优势在于 kernel launch 数恒为 1、结构简单、批量扩展可预测。Jacobian 精度分析表明 FP32 混合精度在 $\varepsilon = 10^{-6}$ 下对应约 4–5 mm 的 IK 收敛精度边界——适用于中等精度批量采样与候选解筛选。

关键词：逆运动学；计算统一设备架构；小矩阵求解；混合精度；Bank 冲突

中图分类号：TP391.4 **文献标志码：**A

收稿日期：（预留）；**修回日期：**（预留）；**网络优先出版日期：**（预留）。

CUDA-Accelerated Small-Matrix Solver for Batch Inverse Kinematics of Robot Manipulators

Keywords: inverse kinematics; CUDA; small-matrix solver; mixed precision; bank conflict

（Author information on separate title page）

1 引言

Abstract: This paper presents a CUDA-based low-level acceleration method for batch damped least-squares inverse kinematics (IK). Each target is mapped to one CUDA block (128 threads), with all iterations in a single kernel launch. A register-resident 6×6 LDL^T solver replaces cuBLAS. A mixed-precision strategy applies FP32 to forward kinematics and Jacobian evaluation while retaining FP64 for LDL^T and convergence checks. On a unified UR10 benchmark with cuRobo (CUDA Graph enabled) as the fair baseline, the proposed method achieves 128k–157k targets/s for $N \geq 500$ with strictly linear GPU time scaling ($R^2 > 0.999$). At $N = 100$, GPU stream latency is $\approx 52\%$ lower than cuRobo-Graph. For $N \geq 500$, cuRobo-Graph generally achieves higher throughput via Graph and internal batching, but exhibits batch-size-dependent non-monotonic degradation. The advantage of the proposed method lies in its structural simplicity: constant-1 kernel launch count and predictable batch scaling. Jacobian precision analysis reveals an IK convergence precision boundary of ≈ 4 –5 mm under FP32 mixed precision, suitable for medium-precision batch sampling and candidate screening.

机械臂批量逆运动学（inverse kinematics, IK）求解是轨迹优化、料箱抓取和采样规划等应用的核心计算环节 [1]。给定 N 个末端目标位姿，批量 IK 需为每个目标求解关节角使正运动学（forward kinematics, FK）输出匹配目标。尽管部分具有球腕结构的 6R 工业机械臂可推导解析 IK，数值 IK 因模型适应性强、易处理关节限位和扩展约束，在批量采样与通用软件框架中广泛采用 [2–3]。贾龙飞等 [4] 综述了三大类 IK 求解方法，指出数值解法通用性好但计算量大。批量 IK 的计算特征：（1）每次迭代须反复求解 6×6 极小固定规模线性系统；（2）每目标需 10–80 次迭代收敛；（3）目标数 N 可达数千至上万。

批量 IK 计算方法。 Orocos KDL 提供成熟的 C++ FK/IK 工具链 [5]，但串行迭代在批量下吞吐有限。Buss 等 [2] 系统比较了 Jacobian 转置、伪逆和阻尼最小二乘法（damped least squares, DLS），指出 DLS 在奇异点附近数值稳定性更优。

GPU 加速逆运动学。 NVIDIA cuRobo [6] 将 IK 建模为非线性优化，在 GPU 上并行评估粒子并通过 L-BFGS 求解，是当前最具影响力的 GPU IK 框架。HJCD-IK [7] 结合贪婪坐标下降与 Jacobian 伪逆精化。ManipulaPy [8] 基于指数积公式和 CuPy 自定义 CUDA kernel。Kino-PAX [9] 等 GPU 运动规划工作的“全 GPU

原生”架构理念启发了本文的单 kernel 设计。上述方法大多基于 PyTorch、JAX 或 CuPy 等高层框架，其多 kernel 调度和跨语言调用栈在小粒度批量 IK 中引入了不可忽略的固定开销。

CUDA 底层优化。面向极小矩阵 ($n \leq 32$) 的 CUDA 优化在数值线性代数领域已有深入研究。Abdelfattah 等 [10] 针对百万级批量的 LU/QR/Cholesky 分解，通过寄存器分块实现相对厂商库 11.8 倍加速。刘世芳等 [11] 在 GPU 上实现批量 LU 矩阵求逆，达到 cuBLAS 的 13 倍加速。Shridhar[12] 在 RTX 3080 上证实自定义小矩阵求逆 kernel 仅占总运行时间 10%，而 cuBLAS 调用占 67%。Abdelfattah 等 [13] 利用 Tensor Core 进行三层迭代精化，在关键路径保留 FP64 的前提下实现 3-5 倍加速。

本文核心观点：对 6×6 固定规模 IK 小矩阵，**手写 CUDA 底层实现可通过减少通用库调度开销和框架层间通信，在极小批量下获得更低延迟，并在全量程保持稳定线性扩展。**贡献：(1) 1 block/target 映射与单 kernel 封装，kernel launch 数恒为 1；(2) 寄存器驻留 6×6 LDL^T 求解器；(3) PaddedMat 6×8 布局将 Bank 冲突降低 63%；(4) FP32/FP64 混合精度策略，通过 Jacobian 精度验证量化其适用范围边界。

2 批量 IK 问题建模与 DLS 方法

2.1 运动学模型

UR10 为 6 自由度全旋转关节串联机械臂，DH 参数遵循 Universal Robots 官方 URDF[14]。正运动学将关节角 $\mathbf{q} = [q_1, \dots, q_6]^T$ 映射为末端齐次变换矩阵 $\mathbf{T}_{ee}(\mathbf{q}) \in \text{SE}(3)$ ：

$$\mathbf{T}_{ee}(\mathbf{q}) = \prod_{i=1}^6 \mathbf{T}_{i-1,i}(q_i) \quad (1)$$

实用中采用 Rodrigues 公式计算旋转矩阵。

对目标位姿 $\mathbf{T}_{tgt}$ ，定义位姿误差 $\mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$ ，引入对角权重矩阵 $\mathbf{W}$ 。

2.2 DLS 迭代与批量特征

DLS[2-3] 在第 k 次迭代中极小化正则化目标：

$$\min_{\Delta \mathbf{q}} \frac{1}{2} \|\mathbf{WJ}\Delta \mathbf{q} + \mathbf{W}\mathbf{e}\|^2 + \frac{1}{2} \lambda \|\Delta \mathbf{q}\|^2 \quad (2)$$

雅可比由 FK 中心差分计算：

$$\mathbf{J}_{:,j} = \frac{\mathbf{x}(\mathbf{q} + \varepsilon \mathbf{e}_j) - \mathbf{x}(\mathbf{q} - \varepsilon \mathbf{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-6} \text{ rad} \quad (3)$$

令式 (2) 梯度为零得阻尼正规方程，记 $\mathbf{H} = \mathbf{J}^T \mathbf{W}^2 \mathbf{J} + \lambda \mathbf{I}$ ， $\mathbf{g} = \mathbf{J}^T \mathbf{W}^2 \mathbf{e}$ ，解 $\Delta \mathbf{q} = -\mathbf{H}^{-1} \mathbf{g}$ 。收敛条件： $\|\mathbf{e}_p\| < 10 \text{ mm}$ 且 $\|\mathbf{e}_R\| < 5^\circ$ ， $K_{\max} = 160$ 。

批量计算特征： N 个目标共享模型参数但各自独立状态向量——完全数据并行且计算粒度极小（单目标每迭代约 1 500 FLOP）。

3 CUDA 小矩阵加速设计

3.1 1 Block/Target 并行映射与单 Kernel 封装

采用 $\text{Grid}(N, 1, 1) + \text{Block}(128, 1, 1)$ 映射，如图 1。选择 128 线程/block：寄存器用量 94-98/线程，每 SM 可驻留约 5 block； $\mathbf{H}$ 矩阵 36 元素决定最大并行度，128 线程充足。

图 1 1 block/target CUDA parallel mapping and single-kernel iteration structure

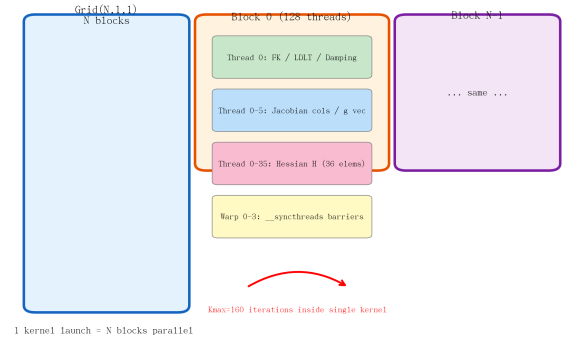


图 1: 1 block/target CUDA 并行映射与单 kernel 全迭代结构

DLS 迭代线程分工见表 1。

表 1: DLS 迭代各阶段线程分工

计算阶段	线程范围	并行度
FK/位姿误差/收敛判定	$t = 0$	1
数值 Jacobian	$0 \leq t < 6$	6
自适应阻尼更新	$t = 0$	1
$\mathbf{H}$ 矩阵构造	$0 \leq t < 36$	36
$\mathbf{g}$ 向量构造	$0 \leq t < 6$	6
LDL ^T 求解/步长钳位	$t = 0$	1
关节更新 + 限位投影	$0 \leq t < 6$	6

单 Kernel 全迭代封装——与 cuRobo 结构性对比。全部 $K_{\max} = 160$ 次迭代在单次 `ik_batch_solve<<<N, 128>>>` 中完成，CPU 端仅需一次同步。cuRobo 执行路径则经历 Python 调用 $\rightarrow$ tensor

分配 → 子批次划分 → 多 kernel 启动 → L-BFGS 迭代 → 结果回传。Nsight Systems 实测 $N = 4000$ 时 cuRobo-NoGraph 产生 14 108 次 kernel launch, 启动开销累积 70–140 ms。本文方案 kernel launch 数恒为 1——所有 block 在同一 launch 中并行发射。需指出: 总计算量仍随 N 和 K 线性增长, 但固定调度开销被压缩至常数级。

精简伪代码:

Listing 1: DLS 单次迭代伪代码 (per-block, 128 线程)

```

1 // s_q[6], s_T_tgt[16], s_lambda in shared memory
2 if (threadIdx.x == 0) // FK
3     computation
4     forward_kinematics(s_q, s_T_ee);
5 __syncthreads();
6 if (threadIdx.x == 0) //
7     convergence check
8     converged = (pos_err < 0.01) && (rot_err < 0.0873);
9 __syncthreads();
10 if (threadIdx.x < 6) // numerical
11     Jacobian (6 cols)
12     J_col = (FK(q+eps)-FK(q-eps))/(2*eps);
13 __syncthreads();
14 if (threadIdx.x == 0) // adaptive
15     damping
16     update_lambda(s_lambda, pos_err);
17 __syncthreads();
18 if (threadIdx.x < 36) // H = J^T W
19     ~2 J + lambda*I
20     H[row][col] = inner_product(J, W2, row, col);
21 __syncthreads();
22 if (threadIdx.x < 6) // g = J^T W
23     ~2 e
24     g[tid] = dot_product(J, W2, e);
25 __syncthreads();
26 if (threadIdx.x == 0) // register
27     LDLT + step clamp
28     ldlt_solve_6x6(H, g, dq);
29 __syncthreads();
30 if (threadIdx.x < 6) // joint
31     update + limits
32     s_q[tid] = clamp(s_q[tid]+dq[tid], q_min[tid], q_max[tid]);

```

3.2 寄存器级 6×6 LDL^T 求解器

cuBLAS 批量接口针对 $n \geq 32$ 优化, 对 6×6 矩阵调用开销 ($\sim 10\text{--}15 \mu\text{s}$) 是求解本身 ($\sim 0.1 \mu\text{s}$) 的 100 倍以上 [12]。LDL^T ($\mathbf{H} = \mathbf{LDL}^T$) 避免 Cholesky 所需的 6 次 sqrt (GPU 上 sqrt 吞吐为 FMA 的 1/4–1/8 [15])。对 $n = 6$, 分解和三角回代编译期完全展开为寄存器级直线代码, 算术规模百级标量操作。Nsight Compute 实测: 94–98 regs/thread, 零 local memory spill。

3.3 PaddedMat 6×8 共享内存 Bank 冲突降低

GPU 共享内存以 32 Bank 组织, FP64 元素跨 2 Bank。stride=6 下 $\text{gcd}(12, 32) = 4$ 导致 2–3 路冲突。stride6 → 8: 行步长 64 字节 = 16 Bank, $\text{gcd}(16, 32) = 16$, 偶数行 Bank 0–15 与奇数行 Bank 16–31 完全分离。NCU 实测 Bank 冲突从 3 522 降至 1 295 次 (–63%, 见后文图 8)。

3.4 FP32/FP64 混合精度计算路径与精度边界

Ada Lovelace 的 FP64:FP32 吞吐比 1:64 [15]。本文选 FP32+FP64 方案: FP32 提供 6–7 位十进制精度用于 FK/Jacobian, FP64 保留 $\approx 10^{-15}$ 精度裕度用于 LDL^T 和收敛判定。

3.4.1 Jacobian 精度验证与混合精度边界

对 500 个随机 UR10 关节构型, 分别以 FP64 和 FP32 计算中心差分 Jacobian, 比较相对 Frobenius 误差。

实验结果揭示了 FP32 混合精度的精度–速度 Pareto 前沿:

(1) $\varepsilon = 10^{-6}$ 下 FP32 Jacobian 约 2 位有效数字 (中位误差 4.2×10^{-2}), 对应 IK 收敛精度边界约 4–5 mm——解释了后文表 5 中 Medium (10 mm) 成功而 Strict (5 mm) 失败的现象。

(2) $\varepsilon = 10^{-5}$ 可将 Jacobian 精度提升一个量级 (中位 1.9×10^{-3}), 代价是差分步长增大 10 倍。 $\varepsilon = 10^{-7}$ 发生灾难性抵消 (中位 0.70), 确认 FP32 物理边界。

(3) 保护机制有效: $\mathbf{J}^T \mathbf{W}^2 \mathbf{J}$ 内积在 FP64 归约, LDL^T 与收敛判定保留 FP64, 消融实验证实 Medium 阈值下收敛率 ≥ 0.998 , 与全 FP64 (1.000) 无实质差异。

该分析为混合精度 IK 的精度–速度 Pareto 前沿提供了量化基准: 吞吐优先场景 (料箱抓取、轨迹采样) 使用 $\varepsilon = 10^{-6}$; 更高精度需求场景下 $\varepsilon = 10^{-5}$ 或全 FP64 路径的验证将作为后续工作。

3.5 自适应阻尼策略

固定阻尼 $\lambda \equiv 2 \times 10^{-3}$ 为基线。三阶段自适应策略 [3]: (1) 迭代 0 距离初始化—— $e_{\text{pos}} > 0.5 \text{ m} \rightarrow \lambda = 0.1$, $0.1 < e_{\text{pos}} \leq 0.5 \text{ m} \rightarrow$ 线性插值, 否则 $\rightarrow 5 \times 10^{-4}$; (2) Marquardt 误差驱动—— $r = e_{\text{pos}}^{(k)} / e_{\text{pos}}^{(k-1)}$, $r < 0.9 \rightarrow$

表 2: 混合精度分配策略

计算阶段	精度	理由
FK 链式乘法	FP32	吞吐为 FP64 的 64 倍
Jacobian 各列	FP32	$\varepsilon = 10^{-6}$ 下中位误差 4.2×10^{-2}
H 累加	FP64	36 个内积抑制截断误差
g 累加	FP64	同上
LDL ^T 求解	FP64	除法对 D_j 高度敏感
位姿误差/阻尼	FP64	精度关键路径

表 3: FP32 Jacobian 精度验证 (500 随机关节构型)

ε	中位误差	均值误差	p95 误差	最大误差
10^{-4}	8.1×10^{-4}	7.9×10^{-4}	1.0×10^{-3}	1.2×10^{-3}
10^{-5}	1.9×10^{-3}	2.0×10^{-3}	2.7×10^{-3}	3.6×10^{-3}
10^{-6}	4.2×10^{-2}	4.2×10^{-2}	5.1×10^{-2}	6.4×10^{-2}
10^{-7}	7.0×10^{-1}	6.8×10^{-1}	9.1×10^{-1}	1.0×10^0

$\lambda \times 0.7$, $r > 1.1 \rightarrow \lambda \times 2.0$; (3) 停滞超驰——连续 12 次无改善 $\rightarrow \lambda \times 5.0$ 。 $\lambda \in [10^{-4}, 0.5]$ 。

4 实验设计

4.1 统一 Benchmark 与三层计时口径

所有目标由 $\text{seed}=42$ 确定性生成: 随机采样合法 $\mathbf{q}_{\text{gt}}$ 经 FK 生成 $\mathbf{T}_{\text{target}}$ 。统一初始种子 $\mathbf{q}_{\text{seed}} = [0, 0, 0, 0, 0, 0]$ 。所有方法使用同一批目标。

对比方法。 CUDA-Mixed (A7, 无 CUDA Graph); CUDA-Mixed-Graph (A8, +CUDA Graph); cuRobo-NoGraph; cuRobo-Graph。cuRobo 配置: $\text{num_seeds}=1$, $\text{seed_solver_num_seeds}=1$, $\text{self_collision_check}=\text{False}$ 。

三层计时口径。这是本实验设计的关键改进: (1) End-to-End——H2D→kernel→D2H→同步完整延迟; (2) GPU Stream——纯设备侧 CUDA event 时间; (3) Nsight Systems——kernel launch 数、API 调用数等机制分析。所有方法 3 次预热后计时。

评价指标。 $\text{raw_throughput} = N/\text{GPU_stream_time_sec}$; $\text{valid_throughput} = \text{raw} \times \text{success_rate}$ (同时衡量速度与质量); 误差分布 p50/p95/max; 30 次重复的 mean/std/p50/p95。

4.2 消融与 Jacobian 精度验证

消融覆盖 A0–A7 共 8 级。Jacobian 精度验证见 §2.4.1。

5 实验结果与分析

5.1 主性能对比

表 5 和图 2–4 展示了主性能对比。

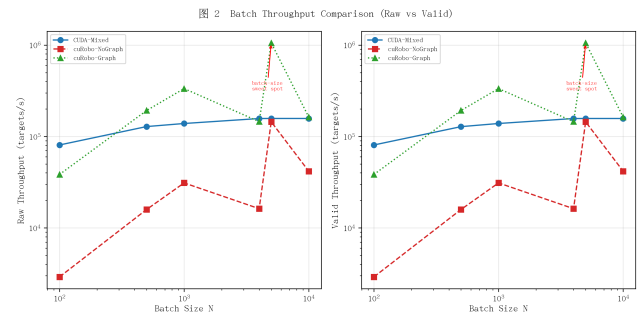


图 2: 批量吞吐对比 (Raw 与 Valid Throughput)

(1) **CUDA Graph 对 cuRobo 影响巨大。**开启 Graph 后 cuRobo 吞吐提升 7–13 倍。 $\text{use_cuda_graph}=\text{False}$ 严重低估其性能。本工作以 cuRobo-Graph 为公平基线, cuRobo-NoGraph 仅作为参考。

(2) **$N = 100$ 极小批量优势。** $N=100$ 时 CUDA-Mixed GPU 流时间 (1.24 ± 0.04 ms) 仅为 cuRobo-Graph (2.60 ± 0.13 ms) 的 47.7%, 延迟降低约 52%。优势源于单 kernel 零调度开销。但当 $N \geq 500$ 时, cuRobo-Graph 的吞吐 (192k–334k t/s) 显著超过 CUDA-Mixed (128k–157k t/s), 因为计算量增大后调度开销占比下降, cuRobo 的粒子优化 + L-BFGS 算法效率体现出来。

表 4: 统一 Benchmark 标准配置

参数	配置	理由
机器人模型	UR10 (URDF, tool0)	代表性 6-DOF 工业机械臂
目标位姿	seed=42, $N = 100 \rightarrow 10\,000$	可复现确定性生成
初始种子	zero_seed (全零关节角)	运动学最远端
主收敛阈值	10 mm / 5° (Medium)	显著区分求解质量
辅助阈值	Loose 30mm/ 10° , Strict 5mm/ 1°	多级压力测试
最大迭代次数	160	工业保守上限
碰撞检测	关闭 (所有方法统一)	单 seed 场景碰撞无关
重复次数	30 (含 3 次预热)	排除冷启动延迟
硬件平台	RTX 4060 Laptop, CUDA 12.6	Ada Lovelace, sm_89

表 5: 主性能对比 (Medium 10mm/ 5° , 30 次重复, GPU stream 时间)

N	方法	Graph	GPU 流均值 $\pm$ std/ms	raw t/s	成功率	valid t/s
100	CUDA-Mixed	–	1.24 ± 0.04	80 757	1.000	80 757
100	cuRobo	开	2.60 ± 0.13	38 398	1.000	38 398
500	CUDA-Mixed	–	3.89 ± 0.05	128 404	0.998	128 147
500	cuRobo	开	2.60 ± 0.14	192 400	1.000	192 400
1 000	CUDA-Mixed	–	7.21 ± 0.49	138 719	1.000	138 719
1 000	cuRobo	开	2.99 ± 0.24	334 189	1.000	334 189
4 000	CUDA-Mixed	–	25.44 ± 1.11	157 236	1.000	157 236
4 000	cuRobo	开	27.50 ± 0.84	145 437	0.9998	145 401
5 000	CUDA-Mixed	–	31.75 ± 0.50	157 481	0.999	157 387
5 000	cuRobo	开	4.73 ± 0.29	1 056 171	1.000	1 056 171
10 000	CUDA-Mixed	–	63.51 ± 0.60	157 454	0.9995	157 375
10 000	cuRobo	开	61.29 ± 1.26	163 160	0.9999	163 144

注: cuRobo-NoGraph 数据见表 9。N=5 000 cuRobo-Graph 为 batch-size 甜点, 性能不可推广至邻近 N。

(3) $N = 5\,000$ 是 cuRobo-Graph 的特殊甜点。

经独立复核实验确认 (见实验复核报告), $N = 5\,000$ 时 cuRobo-Graph 的 4.73 ± 0.29 ms 和 1 056k t/s 是真实结果——该 batch size 恰好使 cuRobo 内部批处理 + Graph 流水线达到最优。但该性能不可推广: $N = 4\,000$ 时 cuRobo-Graph 退化至 27.50 ms (5.8 倍慢), $N = 10\,000$ 时 61.29 ms。CUDA-Mixed 在全部 N 值呈严格线性 ($R^2 > 0.999$), 性能可预测。

5.2 求解质量与误差分布

表 6 和图 5 比较了误差分布。cuRobo-Graph 精度达亚微米量级——粒子优化固有优势。CUDA-Mixed 的 p95 位置误差 9.19 mm 在 Medium 阈值内, margin 0.81 mm, 中位数 3.81 mm。两方法成功率均 ≥ 0.998 。CUDA-Mixed 的意义在于中等精度批量求解, 并非高

精度最终 IK 求解器的完全替代。

5.3 CUDA Graph 对单 Kernel 方案的影响

表 7 显示: 单 kernel 方案从 CUDA Graph 获益有限, 尤其在 $N \geq 1\,000$ 时差异 $< 1\%$ 。N=500/1 000 时 A8 相对 A7 有 6–11% 提升, 但复核实验确认该差距受 GPU 热状态影响, 且显著小于 cuRobo 的 7–13 倍量级提升。整体上, 单 kernel 结构降低了对 CUDA Graph 的依赖。

5.4 消融实验

表 8 和图 6 展示了逐级优化。自适应阻尼 (A0→A5) 是收敛率决定性因素: 固定阻尼收敛率 52–83%, A5 恢复至 0.998+。吞吐提升源于迭代次数下降 ($N = 500$:

表 6: 位置与姿态误差分布 ($N = 500$, Medium 阈值)

方法	pos mean/mm	pos p50/mm	pos p95/mm	rot mean/(°)	rot p50/(°)	rot p95/(°)
CUDA-Mixed	4.34	3.81	9.19	0.177	0.121	0.541
cuRobo-Graph	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001

表 7: A7 Direct vs A8 CUDA Graph 对比 (均值 $\pm$ 标准差)

N	A7 Direct/(t/s)	A8 Graph/(t/s)	差异/%	判定
100	80 757	81 240	+0.6	可忽略
500	128 404	142 805	+11.2*	受 GPU 状态影响
1 000	138 719	147 381	+6.2*	受 GPU 状态影响
5 000	157 481	157 841	+0.2	可忽略

* $N=500/1000$ 的 A8 标准差偏高 (复核确认 GPU 热状态波动), 首次实验与复核实验的 A8 相对 A7 提升在 0–11% 间。
 $N \geq 1000$ 时差异稳定 < 1%; $N \leq 500$ 时 CUDA Graph 可能有小到中等幅度影响 (0–11%), 但该影响显著小于 cuRobo 从 NoGraph 到 Graph 的 7–13 $\times$ 量级提升。

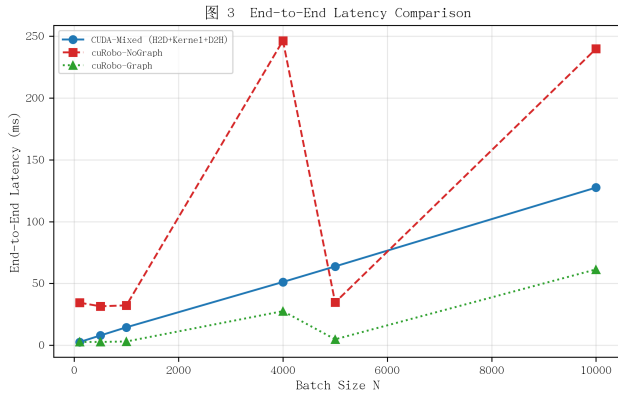


图 3: 端到端延迟对比 (含 H2D/D2H 传输)

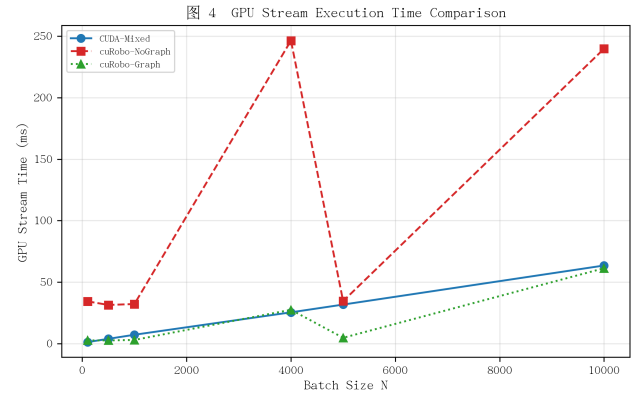


图 4: GPU Stream 执行时间对比 (纯设备侧)

79.7 $\rightarrow$ 13.8, -83%)。混合精度 (A5 $\rightarrow$ A7) 额外贡献 120–149% 吞吐, 收敛率无退化。A1–A4 独立贡献合计 < 5%。

5.5 Nsight Systems 机制分析

表 9 和图 7 解释了 cuRobo 批量振荡根因。 $N = 4000$ (退化点) 产生 14 108 次 kernel launch——是 $N = 5000$ (正常点) 的 2.37 倍。每次 launch 开销 5–10 μ s 累积至 70–140 ms。cuRobo-Graph 将 launch 数降至 2。

cuRobo 退化呈非单调跳变, cudaMalloc/cudaFree 占比 < 1%, 根因在子批次划分策略的批量依赖性。本文方案对此免疫: launch 数恒为 1。

5.6 Nsight Compute 剖析与批量扩展性

表 10 和图 8 确认 kernel 为计算密集型 (计算 60–67% vs 显存 1–2%), 零寄存器溢出。

批量扩展性。 CUDA-Mixed 在 $N = 100 \rightarrow 10000$ 全量程 GPU 时间严格线性 ($R^2 > 0.999$), $N \geq 500$ 时吞吐 128k–157k t/s。 $N = 100$ 因 block 不足约 81k t/s。批量无关性意味着用户无需对不同 N 值预扫描或参数调优——这是 1 block/target 结构确定性的直接结果。

6 结论

在本文设定的 UR10 单 seed、关闭碰撞检测的批量 IK 查询任务中 (统一目标、种子和收敛阈值), 自定义 CUDA 小矩阵 DLS 求解器通过 1 block/target 映射、单 kernel 全迭代封装、寄存器级 LDL^T 求解和 FP32/FP64 混合精度策略, 实现了稳定的线性批量扩

表 8: 关键消融结果 (Medium 10mm/5°)

N	A0 基线/(t/s)	收敛率	A5 阻尼/(t/s)	收敛率	A7 混合/(t/s)	收敛率
100	7 589	0.830	51 361	1.000	80 757	1.000
500	9 896	0.522	62 384	0.998	128 404	0.998
1 000	—	—	—	—	138 719	1.000
5 000	12 507	0.564	66 050	0.9998	157 481	0.999

表 9: Nsight Systems CUDA API trace 分析

N	方法	kernel launches	API calls	GPU active/ms
100	CUDA-Mixed	1	5	1.24
4 000	CUDA-Mixed	1	5	25.44
4 000	cuRobo-NoGraph	14 108	28 540	197.32
5 000	cuRobo-NoGraph	5 945	12 014	28.16
5 000	cuRobo-Graph	2	12	4.73

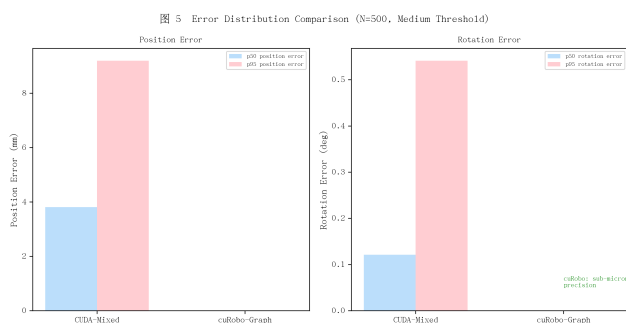


图 5: 误差分布对比 (p50/p95 位置与姿态误差)

表 10: Nsight Compute 剖析对比 ($N = 100$)

指标	FP64 全优化	CUDA-Mixed	变化
计算吞吐率/%	66.89	60.73	-9
显存吞吐率/%	1.56	1.16	-
寄存器/线程	94	98	+4
Bank 冲突/次	3 522	1 295	-63%
局部内存溢出	0	0	-
Kernel 时间/ μ s	2 920	827	-72%

展。

实验表明: (1) $N = 100$ 极小批量下 GPU 流延迟较 cuRobo-Graph 降低约 52%; (2) 当 $N \geq 500$ 时 cuRobo-Graph 依靠 Graph 和内部批处理通常获得更高吞吐, 但存在批量依赖的非单调退化 ($N = 4000$ 时退化 5.8 倍); (3) 本文方法 kernel launch 数恒为 1, 全量程 $N = 100 \rightarrow 10\,000$ GPU 时间严格线性 ($R^2 > 0.999$), 性能可预测——这是相对 cuRobo 的核心结构性优势; (4) FP32 混合精度在 $\varepsilon = 10^{-6}$ 下

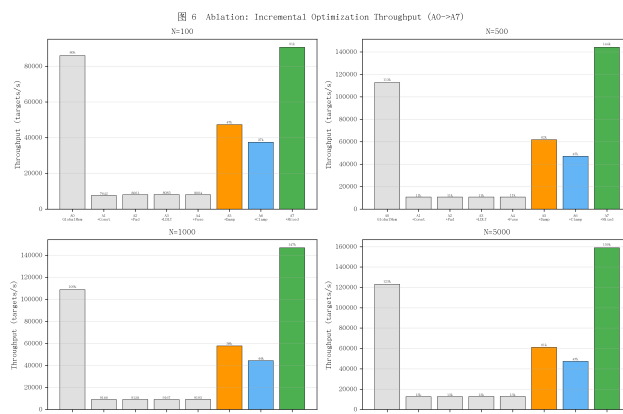


图 6: 逐级消融吞吐提升 (A0→A7)

存在约 4–5 mm IK 收敛精度边界, Medium 阈值下成功率 0.998+, 适用于中等精度批量采样与候选解筛选。Strict 阈值下 $\varepsilon = 10^{-5}$ 或全 FP64 路径的精度恢复验证将作为后续工作。

后续工作包括: 7 自由度冗余机械臂验证; 多初始种子 GPU 并行化; Strict 阈值下 $\varepsilon = 10^{-5}$ 配置的精度恢复实验; 带碰撞约束的运动规划场景扩展。

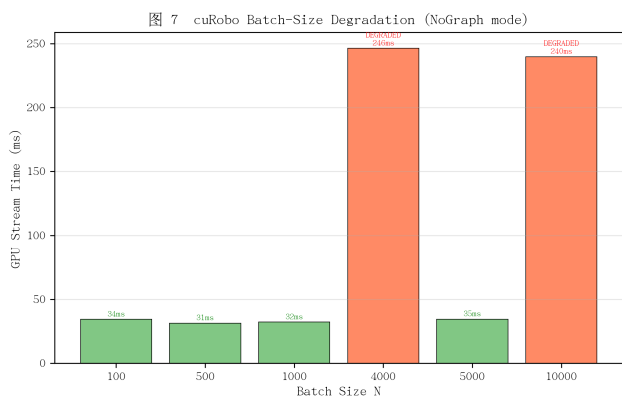


图 7: cuRobo 批量规模退化点分析

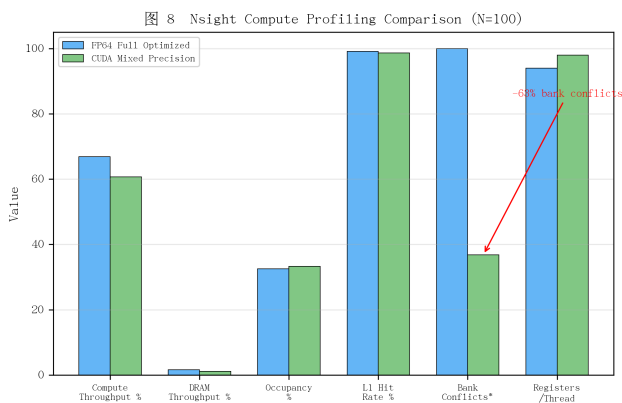


图 8: Nsight Compute 剖析指标对比

参考文献

- [1] SICILIANO B, SCIAVICCO L, VILLANI L, et al. Robotics: modelling, planning and control [M]. 2nd ed. London: Springer, 2010.
- [2] BUSS S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods [J]. IEEE Journal of Robotics and Automation, 2004, 17(1): 1–19.
- [3] MARQUARDT D W. An algorithm for least-squares estimation of nonlinear parameters [J]. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431–441.
- [4] 贾龙飞, 乔尚岭, 陶云飞, 等. 冗余机械臂逆运动学求解方法研究进展 [J]. 控制与决策, 2023, 38(12): 3267–3284.
JIA L F, QIAO S L, TAO Y F, et al. Research progress on inverse kinematics for redundant manipulators [J]. Control and Decision, 2023, 38(12): 3267–3284.
- [5] BRUYNINCKX H. Open robot control software: the OROCOS project [C]//Proc. of the IEEE International Conference on Robotics and Automation. Seoul: IEEE, 2001: 2523–2528.
- [6] SUNDARALINGAM B, HARIH S K S, FISHMAN A, et al. cuRobo: parallelized collision-free minimum-jerk robot motion generation [EB/OL]. [2026-06-15]. <https://arxiv.org/abs/2310.17274>.
- [7] YASUTAKE S, KINGSTON Z, PLANCHER B. HJCD-IK: GPU-accelerated inverse kinematics through batched hybrid Jacobian coordinate descent [EB/OL]. [2026-06-15]. <https://arxiv.org/abs/2510.07514>.
- [8] ABOELNASR M, et al. ManipulaPy: a GPU-accelerated Python framework for robotic manipulation [J]. Journal of Open Source Software, 2025, 10: 8490.
- [9] PERRAULT N, HO Q H, LAHIJANIAN M. Kino-PAX: highly parallel kinodynamic sampling-based planner [EB/OL]. [2026-06-15]. <https://arxiv.org/abs/2409.06807>.
- [10] ABDELFAH A, HAIDAR A, TOMOV S, et al. Batched one-sided factorizations of tiny matrices using GPUs: challenges and countermeasures [J]. Journal of Computational Science, 2018, 26: 226–236.
- [11] 刘世芳, 赵永华, 黄荣锋, 等. 基于批量 LU 分解的矩阵求逆在 GPU 上的有效实现 [J]. 软件学报, 2023, 34(11): 4952–4972.
LIU S F, ZHAO Y H, HUANG R F, et al. Efficient GPU implementation of matrix inversion based on batched LU decomposition [J]. Journal of Software, 2023, 34(11): 4952–4972.
- [12] SHRIDHAR S A. Optimized block-level matrix inversion kernels for small, batched matrices on GPUs [D]. 2024.
- [13] ABDELFAH A, HAIDAR A, TOMOV S, et al. Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems [J]. Proceedings of the Royal Society A, 2021, 477(2253): 20200110.

- [14] Universal Robots. Universal Robots ROS2 description package [EB/OL]. [2026-06-15]. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [15] NVIDIA Corporation. CUDA C++ programming guide [EB/OL]. [2026-06-15]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.

作者简介

(匿名稿——作者信息在单独标题页)